🚀 COURSE NAME: DevOps Bootcamp

🎯 Section Title: Entering Learning Phase 3 - DevOps Core
🧱 Main Course Section: 9 - AWS Services
🧪 Lab Type: **7 - Introduction to EC2 Virtual Cloud Server**

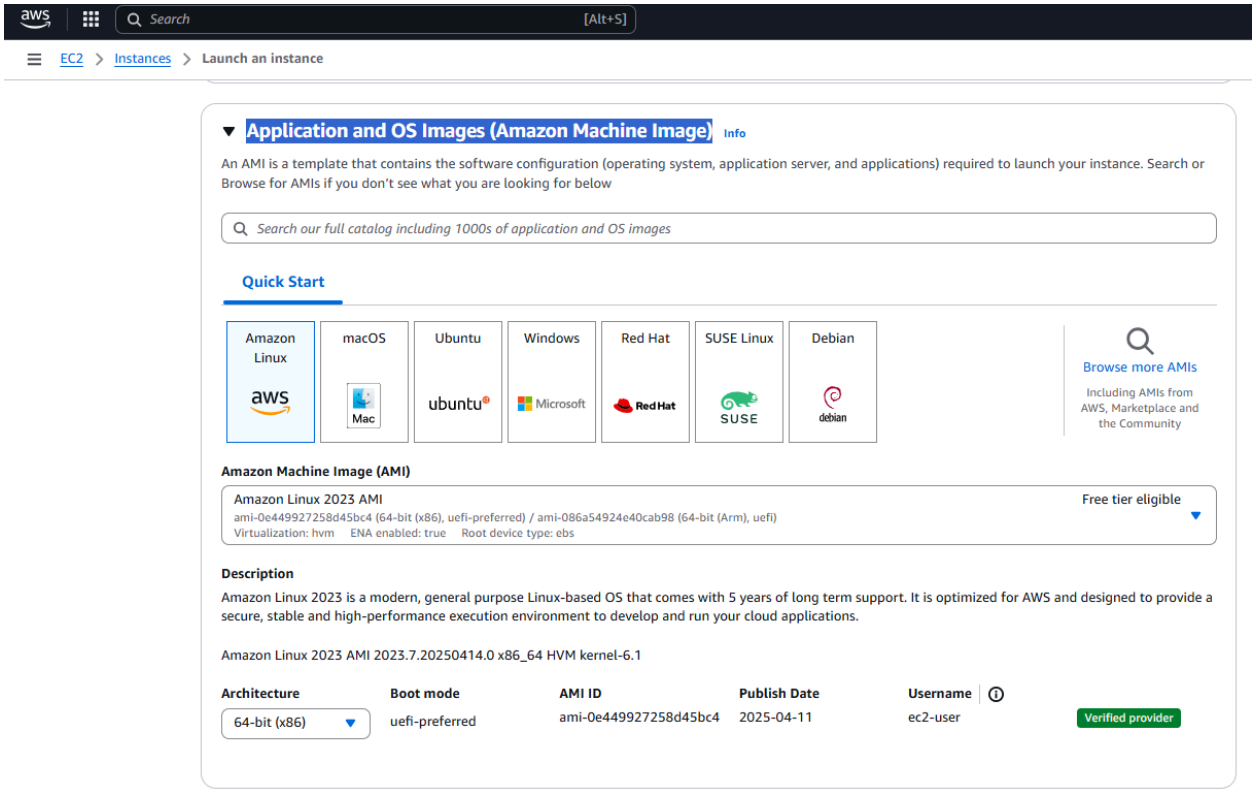
🗃️ Step 1: EC2 Instance Launch – Initial Configuration (AMI, Instance Type, Key Pair, and Networking)

🔹 **Step Overview:** In this step, we walk through the process of launching an EC2 instance, where we configure the instance's Amazon Machine Image (AMI), choose an instance type, configure an SSH key pair for secure login, and set up networking details such as VPC and subnet. This is a crucial step in provisioning an EC2 instance for use in the cloud environment.

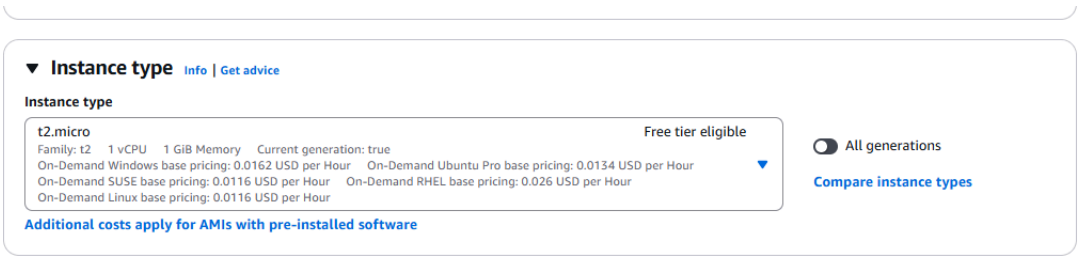
⚙️ Commands Executed:

📁 **Name and Tags:** In this section, we define the instance name and add any relevant tags for identification, making it easier to manage the instance within the AWS environment.

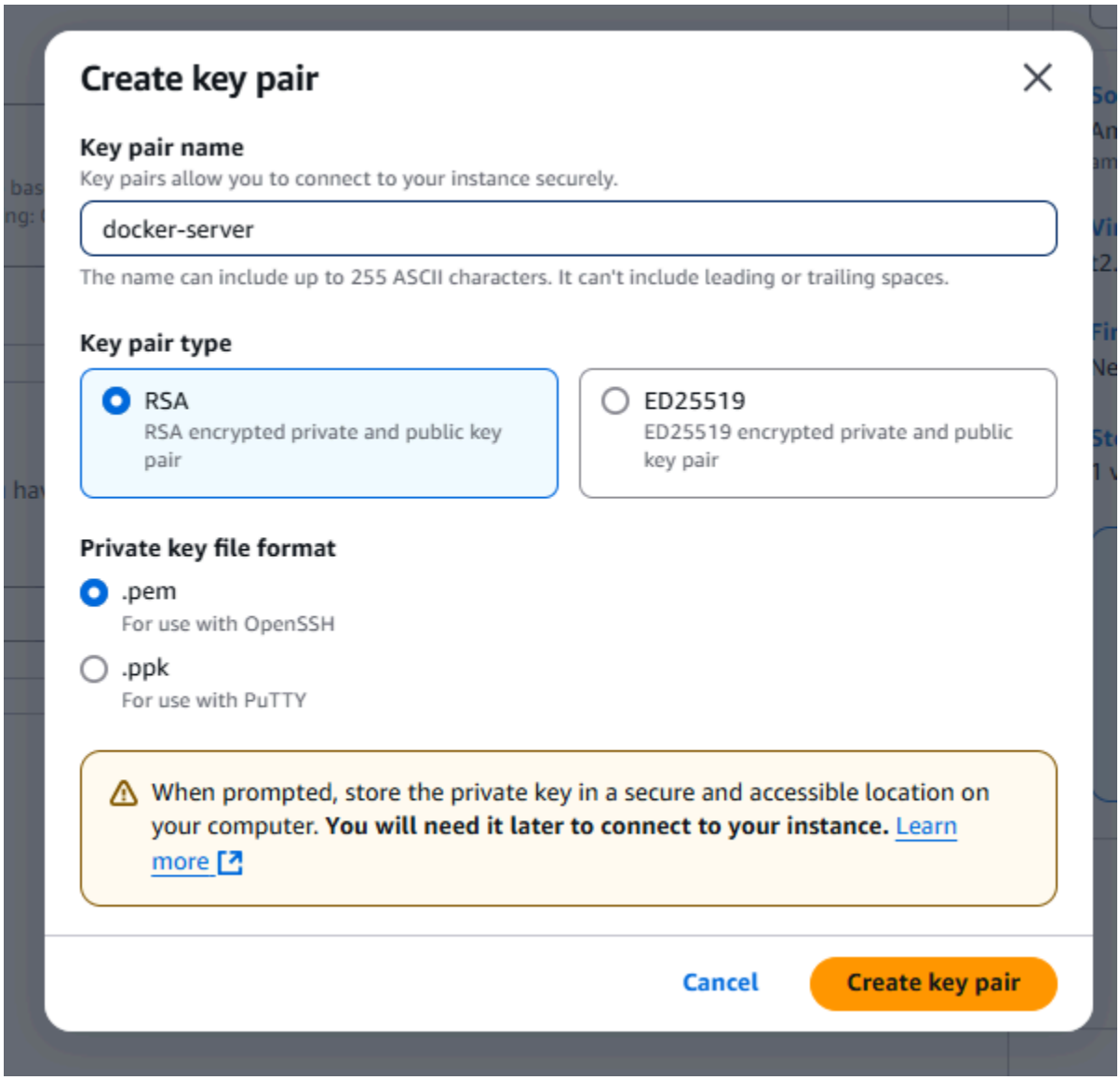
📂 **Application and OS Images (Amazon Machine Image - AMI):** We select an appropriate AMI that contains the operating system and application software required for the instance. This step is essential as it defines the software environment for the instance.



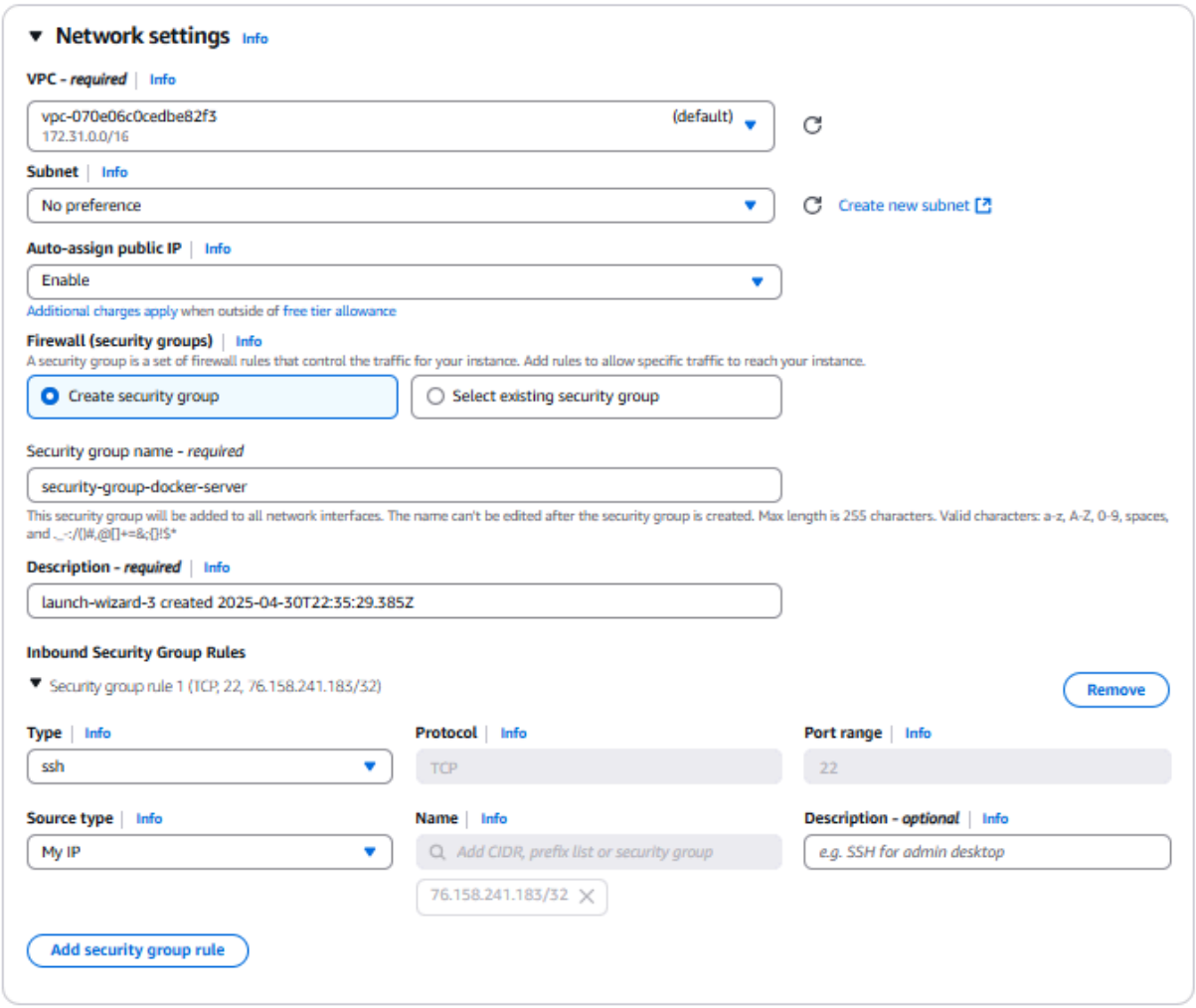
📂 **Instance Type:** The instance type is selected based on the computing resources required (such as CPU, RAM, and storage). In this case, we would select an instance type suitable for the expected workload.



📁 **Key Pair (Login):** Here, we specify an SSH key pair to ensure secure access to the EC2 instance. The key pair allows us to SSH into the instance securely without a password, using private/public key encryption.



📂 **Network Settings:** This step involves configuring the networking parameters for the instance, such as selecting the VPC (Virtual Private Cloud), subnet, and security groups. These settings control the instance's network accessibility and security.



🔍 **What We Learned:**

- Understanding how to set the instance's basic configurations (name, AMI, instance type, key pair, networking).
- Gained insight into how networking and security settings (VPC, subnet, security group) determine the instance's accessibility.
- Recognized the importance of selecting the right AMI and instance type for optimal performance.

✅ **Summary:** You've just launched and configured an EC2 instance with key settings including AMI, instance type, key pair, and networking. This step is crucial in setting up the foundation for your EC2 instance to run applications in AWS securely and efficiently.

🗃️ Step 2: Configuring SSH Key Permissions and Connecting to EC2 Instance

🔹 **Step Overview:** In this step, we configure the SSH key permissions to ensure secure access to the EC2 instance. After moving the key to the appropriate location, we change its permissions for security and connect to the EC2 instance via SSH using the private key. This is essential for establishing a secure connection and managing the EC2 instance remotely.

⚙️ **Commands Executed:**

📁 **Move the Key to the SSH Directory:**

bash
CopyEdit
```
mv Downloads/docker-server.pem ~/.ssh/
```

💡 This command moves the private SSH key to the `~/.ssh/` directory, a secure location for SSH keys on your system.

📂 **Verify Key Permissions:**

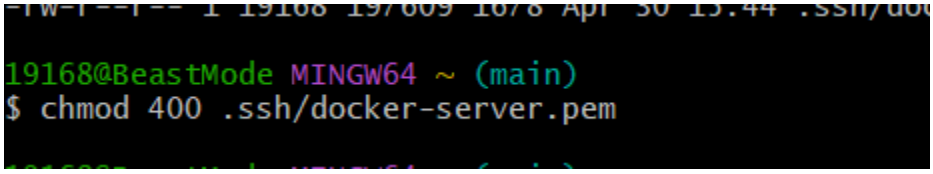
bash
CopyEdit
```
ls -l .ssh/docker-server.pem
```

💡 This command lists the file permissions of the SSH key, allowing us to check that it's placed correctly in the `.ssh` directory.

📂 **Set Secure Permissions for the Key:**

bash
CopyEdit

```
chmod 400 .ssh/docker-server.pem
```

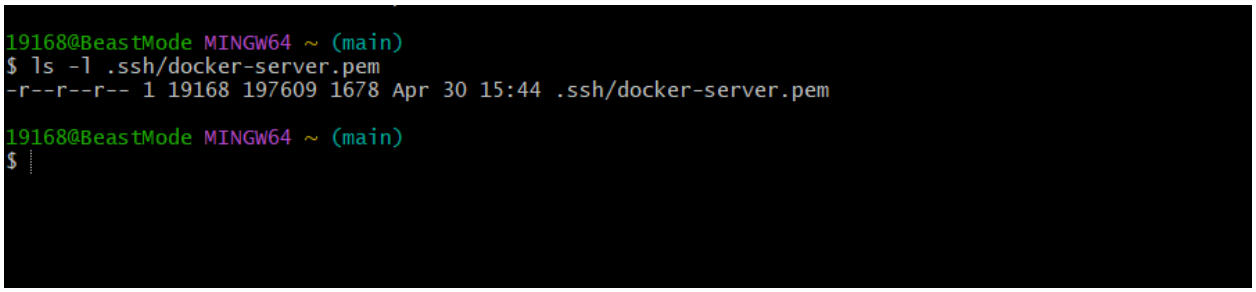


💡 This command sets the permissions of the SSH key file to be read-only for the owner, ensuring that it's secure and can't be altered by other users.

📁 **Verify Permissions After Modification:**

bash
CopyEdit

```
ls -l .ssh/docker-server.pem
```

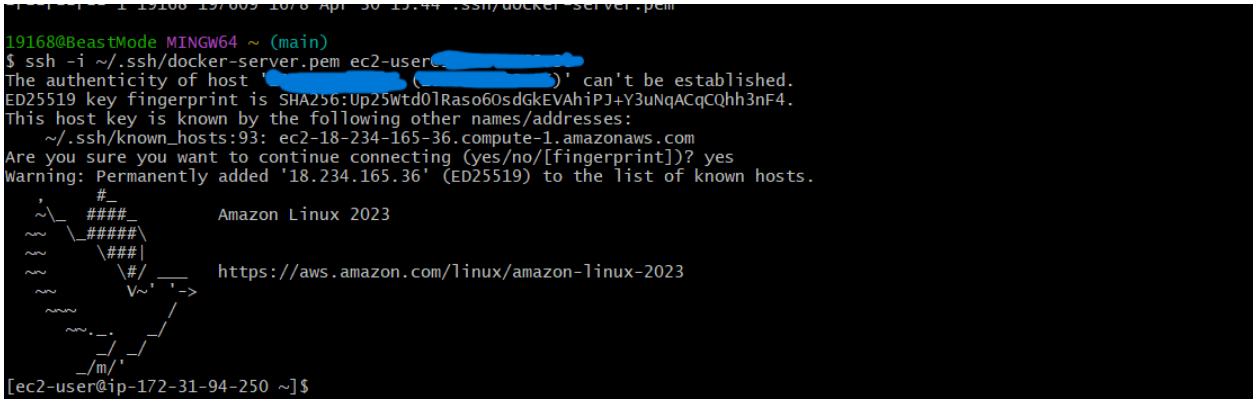


💡 This command rechecks the file permissions to confirm that they've been properly applied, ensuring the key is secure.

📂 **SSH into the EC2 Instance:**

bash
CopyEdit

```
ssh -i ~/.ssh/docker-server.pem ec2-user@publicip
```



💡 This command initiates an SSH connection to the EC2 instance using the private key for authentication, allowing secure access to the instance's terminal.

🔍 **What We Learned:**

- Gained an understanding of how to securely manage SSH key permissions by moving the key to the correct directory and restricting its permissions.
- Learned how to verify the key permissions before connecting to the instance.
- Successfully connected to the EC2 instance using the SSH key, demonstrating how to access a remote server securely.

✅ **Summary:** You've just configured the SSH key for secure access to your EC2 instance and successfully connected to it using the SSH protocol. This step ensures that your communication with the EC2 instance remains encrypted and secure.

🐳 Step 3: Install & Run Docker on EC2 – Setup, Start Service, and Launch Redis Container

🔹 **Step Overview:**

In this step, we installed Docker on the EC2 instance, started the Docker service, added the current user to the Docker group, and successfully ran a Redis container. This process prepares the EC2 instance to run containerized applications and validates Docker's functionality.

⚙️ Commands Executed:

```
19168@BeastMode MINGW64 ~ (main)
$ ssh -i ~/.ssh/docker-server.pem ec2-user[redacted]
The authenticity of host '[redacted] ([redacted])' can't be established.
ED25519 key fingerprint is SHA256:Up25WtdOlRaso6OsdGkEVAhiPJ+Y3uNqACqCQhh3nF4.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:93: ec2-18-234-165-36.compute-1.amazonaws.com
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.234.165.36' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-94-250 ~]$
```

📁 Update the System Packages:

bash
CopyEdit

```
sudo yum update
```

```
ec2-user@ip-172-31-94-250:~
[ec2-user@ip-172-31-94-250 ~]$ yum update
Error: This command has to be run with superuser privileges (under the root user on most systems).
[ec2-user@ip-172-31-94-250 ~]$ sudo yum update
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-94-250 ~]$ |
```

💡 Ensures all system packages are up to date before installing Docker, preventing conflicts and ensuring compatibility.

📂 Install Docker:

bash
CopyEdit

```
sudo yum install docker
```

```
ec2-user@ip-172-31-94-250:~
[ec2-user@ip-172-31-94-250 ~]$ sudo yum install docker
Last metadata expiration check: 0:01:00 ago on Wed Apr 30 23:13:13 2025.
Dependencies resolved.
================================================================================================================================================
 Package                                  Architecture                    Version                                 Repository                    Size
================================================================================================================================================
Installing:
 docker                                   x86_64                          25.0.8-1.amzn2023.0.2                   amazonlinux                   45 M
Installing dependencies:
 containerd                               x86_64                          1.7.27-1.amzn2023.0.2                   amazonlinux                   37 M
 iptables-libs                            x86_64                          1.8.8-3.amzn2023.0.2                    amazonlinux                  401 k
 iptables-nft                             x86_64                          1.8.8-3.amzn2023.0.2                    amazonlinux                  183 k
 libcgroup                                x86_64                          3.0-1.amzn2023.0.1                      amazonlinux                   75 k
 libnetfilter_conntrack                   x86_64                          1.0.8-2.amzn2023.0.2                    amazonlinux                   58 k
 libnfnetlink                             x86_64                          1.0.1-19.amzn2023.0.2                   amazonlinux                   30 k
 libnftnl                                 x86_64                          1.2.2-2.amzn2023.0.2                    amazonlinux                   84 k
 pigz                                     x86_64                          2.5-1.amzn2023.0.3                      amazonlinux                   83 k
 runc                                     x86_64                          1.2.4-1.amzn2023.0.1                    amazonlinux                  3.4 M

Transaction Summary
================================================================================================================================================
Install  10 Packages

Total download size: 86 M
Installed size: 324 M
Is this ok [y/N]: y
Downloading Packages:
(1/10): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.rpm                                                   3.1 MB/s | 401 kB     00:00
(2/10): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rpm                                                    6.3 MB/s | 183 kB     00:00
(3/10): libcgroup-3.0-1.amzn2023.0.1.x86_64.rpm                                                         1.3 MB/s |  75 kB     00:00
(4/10): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64.rpm                                          1.7 MB/s |  58 kB     00:00
(5/10): libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64.rpm                                                   755 kB/s |  30 kB     00:00
(6/10): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm                                                        2.6 MB/s |  84 kB     00:00
(7/10): pigz-2.5-1.amzn2023.0.3.x86_64.rpm                                                              2.0 MB/s |  83 kB     00:00
(8/10): containerd-1.7.27-1.amzn2023.0.2.x86_64.rpm                                                      30 MB/s |  37 MB     00:01
(9/10): runc-1.2.4-1.amzn2023.0.1.x86_64.rpm                                                            3.9 MB/s | 3.4 MB     00:00
(10/10): docker-25.0.8-1.amzn2023.0.2.x86_64.rpm                                                         30 MB/s |  45 MB     00:01
------------------------------------------------------------------------------------------------------------------------------------------------
Total                                                                                                    56 MB/s |  86 MB     00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                                                                        1/1
  Installing       : runc-1.2.4-1.amzn2023.0.1.x86_64                                                                                      1/10
  Installing       : containerd-1.7.27-1.amzn2023.0.2.x86_64                                                                               2/10
  Running scriptlet: containerd-1.7.27-1.amzn2023.0.2.x86_64                                                                               2/10
  Installing       : pigz-2.5-1.amzn2023.0.3.x86_64                                                                                        3/10
  Installing       : libnftnl-1.2.2-2.amzn2023.0.2.x86_64                                                                                  4/10
  Installing       : libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64                                                                             5/10
  Installing       : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64                                                                    6/10
  Installing       : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64                                                                             7/10
  Installing       : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64                                                                              8/10
  Running scriptlet: iptables-nft-1.8.8-3.amzn2023.0.2.x86_64                                                                              8/10
  Installing       : libcgroup-3.0-1.amzn2023.0.1.x86_64                                                                                   9/10
  Running scriptlet: docker-25.0.8-1.amzn2023.0.2.x86_64                                                                                   10/10
  Installing       : docker-25.0.8-1.amzn2023.0.2.x86_64                                                                                   10/10
  Running scriptlet: docker-25.0.8-1.amzn2023.0.2.x86_64                                                                                   10/10
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket - /usr/lib/systemd/system/docker.socket.

  Verifying        : containerd-1.7.27-1.amzn2023.0.2.x86_64                                                                               1/10
  Verifying        : docker-25.0.8-1.amzn2023.0.2.x86_64                                                                                   2/10
  Verifying        : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64                                                                             3/10
  Verifying        : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64                                                                              4/10
  Verifying        : libcgroup-3.0-1.amzn2023.0.1.x86_64                                                                                   5/10
  Verifying        : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64                                                                    6/10
  Verifying        : libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64                                                                             7/10
  Verifying        : libnftnl-1.2.2-2.amzn2023.0.2.x86_64                                                                                  8/10
  Verifying        : pigz-2.5-1.amzn2023.0.3.x86_64                                                                                        9/10
  Verifying        : runc-1.2.4-1.amzn2023.0.1.x86_64                                                                                     10/10

Installed:
  containerd-1.7.27-1.amzn2023.0.2.x86_64          docker-25.0.8-1.amzn2023.0.2.x86_64                  iptables-libs-1.8.8-3.amzn2023.0.2.x86_64          iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
  libcgroup-3.0-1.amzn2023.0.1.x86_64              libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64   libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64          libnftnl-1.2.2-2.amzn2023.0.2.x86_64
  pigz-2.5-1.amzn2023.0.3.x86_64                   runc-1.2.4-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-94-250 ~]$
```

💡 Installs the Docker engine on the EC2 instance, enabling the ability to build and run containers.

📂 **Verify Docker Installation:**

bash
CopyEdit

```
docker -v
```

```
ec2-user@ip-172-31-94-250:~
[ec2-user@ip-172-31-94-250 ~]$ docker -v
Docker version 25.0.8, build 0bab007
[ec2-user@ip-172-31-94-250 ~]$
```

💡 Checks the installed Docker version to confirm that Docker was successfully installed.

📁 **Start Docker Service:**

bash
CopyEdit

```
sudo service docker start
```

```
ec2-user@ip-172-31-94-250:~
[ec2-user@ip-172-31-94-250 ~]$ sudo service docker start
Redirecting to /bin/systemctl start docker.service
[ec2-user@ip-172-31-94-250 ~]$
```

💡 Starts the Docker daemon, which is required to run containers and manage Docker resources.

📂 **Verify Docker Process is Running:**

bash
CopyEdit
```
ps aux | grep docker
```

```
[ec2-user@ip-172-31-94-250 ~]$ ps aux | grep docker
root       27670  0.2  8.4 1773712 82112 ?       Ssl  23:15   0:00 /usr/
bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --defa
ult-ulimit nofile=32768:65536
ec2-user   27822  0.0  0.2 222316  2068 pts/0    S+   23:16   0:00 grep
--color=auto docker
[ec2-user@ip-172-31-94-250 ~]$ |
```

💡 Confirms that the Docker daemon is active and running in the background.

📂 **Add User to Docker Group:**

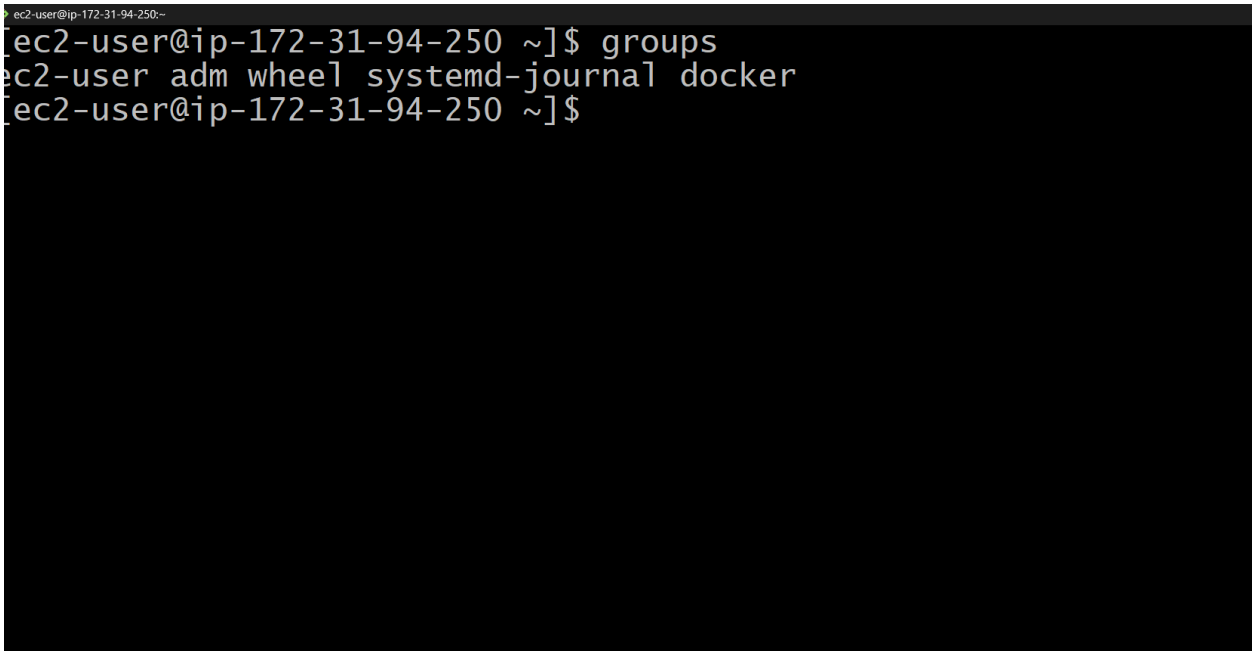
bash
CopyEdit
```
sudo usermod -aG docker $USER
```

```
ec2-user@ip-172-31-94-250:~
[ec2-user@ip-172-31-94-250 ~]$ sudo usermod -aG docker $USER
[ec2-user@ip-172-31-94-250 ~]$ |
```

💡 Adds the current user to the Docker group to allow running Docker commands without `sudo`.

📂 **Check User Groups:**

bash
CopyEdit
```
groups
```



💡 Verifies that the user has been successfully added to the Docker group.

📂 **Run Redis Container:**

bash
CopyEdit

```
docker run redis
```

```
ec2-user@ip-172-31-94-250:~
[ec2-user@ip-172-31-94-250 ~]$ docker run redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
254e724d7786: Pull complete
a131b527c2eb: Pull complete
e14bd717af36: Pull complete
1d380dfb74a4: Pull complete
c9e771e39c60: Pull complete
3781e1e8cbc4: Pull complete
4f4fb700ef54: Pull complete
f5340b1d1605: Pull complete
Digest: sha256:7df1eeff67eb0ba84f6b9d2940765a6bb1158081426745c185a03b150
7de6a09
Status: Downloaded newer image for redis:latest
1:C 30 Apr 2025 23:20:07.212 # WARNING Memory overcommit must be enabled
! Without it, a background save or replication may fail under low memory
 condition. Being disabled, it can also cause failures without low memor
y condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fi
x this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then
 reboot or run the command 'sysctl vm.overcommit_memory=1' for this to t
ake effect.
1:C 30 Apr 2025 23:20:07.213 # WARNING Your system is configured to use
the 'xen' clocksource which might lead to degraded performance. Check th
e result of the [slow-clocksource] system check: run 'redis-server --che
ck-system' to check if the system's clocksource isn't degrading performa
nce.
1:C 30 Apr 2025 23:20:07.214 * oO0OoO0OoO0Oo Redis is starting oO0OoO0Oo
O0Oo
1:C 30 Apr 2025 23:20:07.214 * Redis version=7.4.3, bits=64, commit=0000
0000, modified=0, pid=1, just started
1:C 30 Apr 2025 23:20:07.214 # Warning: no config file specified, using
the default config. In order to specify a config file use redis-server /
path/to/redis.conf
1:M 30 Apr 2025 23:20:07.214 * monotonic clock: POSIX clock_gettime
1:M 30 Apr 2025 23:20:07.215 * Running mode=standalone, port=6379.
1:M 30 Apr 2025 23:20:07.216 * Server initialized
1:M 30 Apr 2025 23:20:07.216 * Ready to accept connections tcp
```

💡 Launches a Redis container to test Docker functionality and run an in-memory data store application.

🔍 **What We Learned:**

- Installed Docker and validated that it's functioning properly on an EC2 instance.
- Understood how to manage Docker permissions by adding the user to the appropriate group.
- Confirmed Docker can run containers by launching Redis successfully.

✅ **Summary:**
You've just installed and configured Docker on your EC2 instance and ran your first container (Redis). This step demonstrates how to prepare a cloud-hosted server for containerized workloads, a key skill in deploying scalable and portable applications.

🚢 Step 4: Deploy Custom Docker App – Pull Image from Docker Hub & Run on EC2

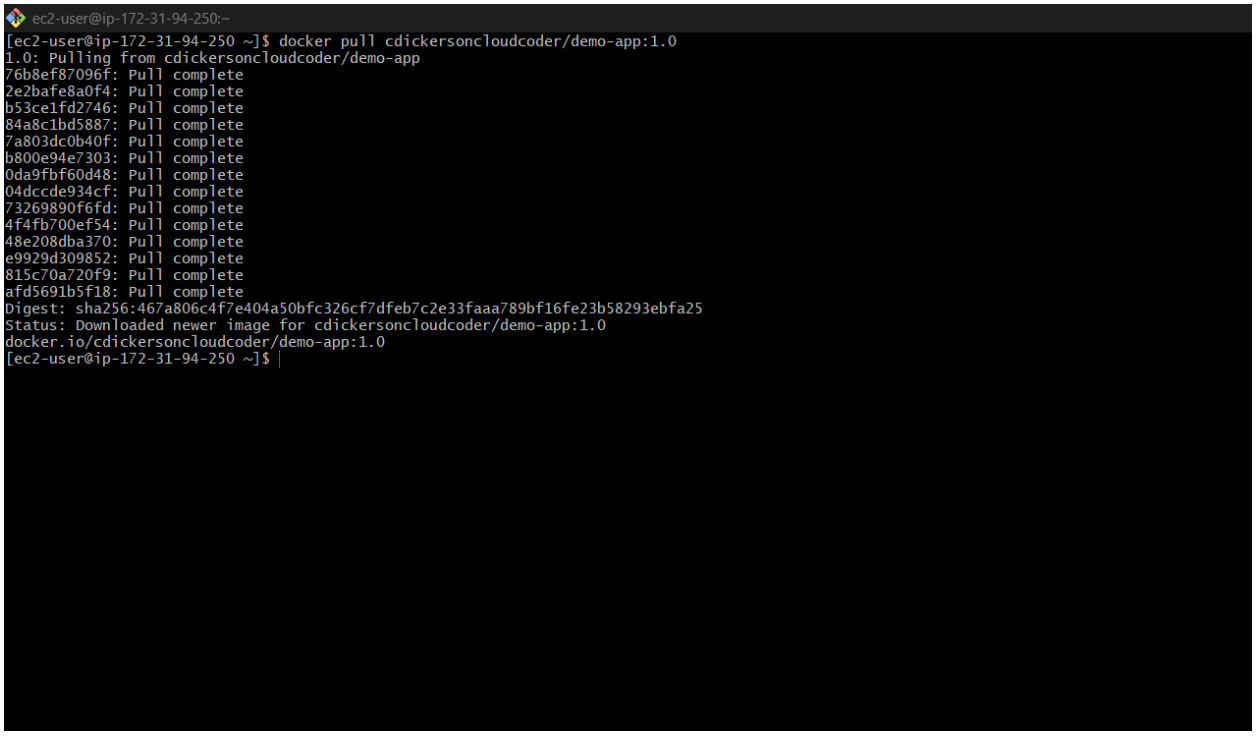
🔹 **Step Overview:**
In this step, we deployed a custom Docker application by pulling a container image from Docker Hub and running it on our EC2 instance. This demonstrates how to deliver applications quickly using container registries and cloud-hosted infrastructure.

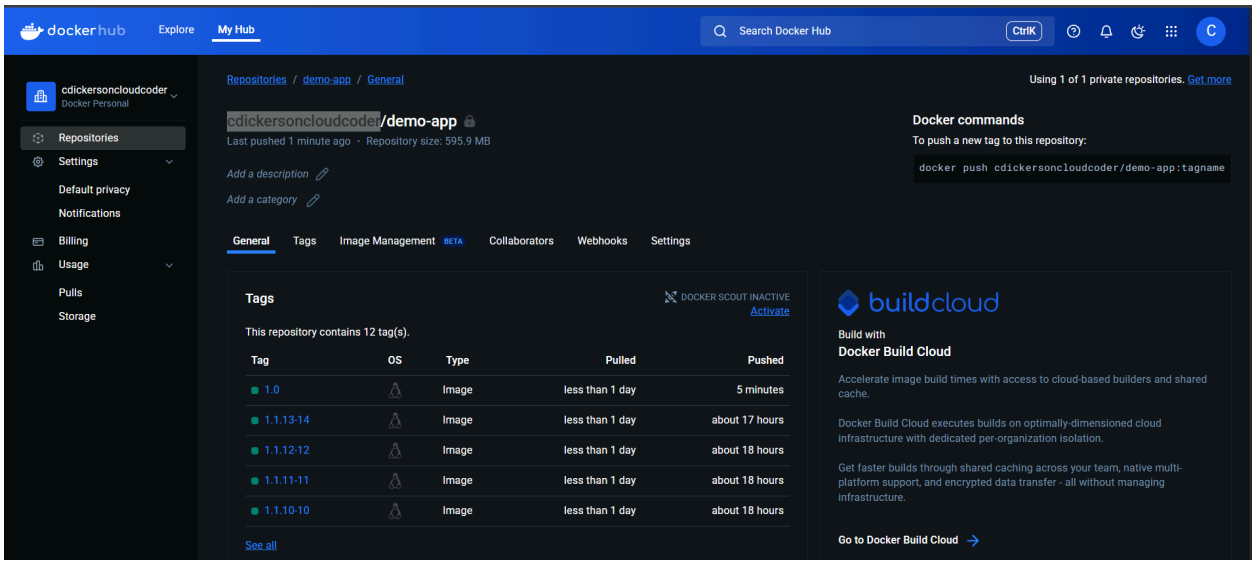
⚙️ **Commands Executed:**

📁 **Pull Custom Image from Docker Hub:**

bash
CopyEdit

```
docker pull cdickersoncloudcoder/demo-app:1.0
```





💡 Retrieves the Docker image `demo-app:1.0` from your public Docker Hub repository, preparing it for deployment on the EC2 instance.

📂 List Available Docker Images:

bash
CopyEdit

```
docker images
```

```
[ec2-user@ip-172-31-94-250 ~]$ docker images
REPOSITORY                    TAG       IMAGE ID       CREATED          SIZE
cdickersoncloudcoder/demo-app 1.0       5f3230c596db   10 minutes ago   950MB
redis                         latest    550dba118f97   7 days ago       117MB
[ec2-user@ip-172-31-94-250 ~]$
```

💡 Displays a list of all Docker images on the system, confirming the `demo-app` image was successfully pulled.

📂 **Run the Custom App Container:**

bash
CopyEdit

```
docker run -p 300:3080 -d cdickersoncloudcoder/demo-app:1.0
```

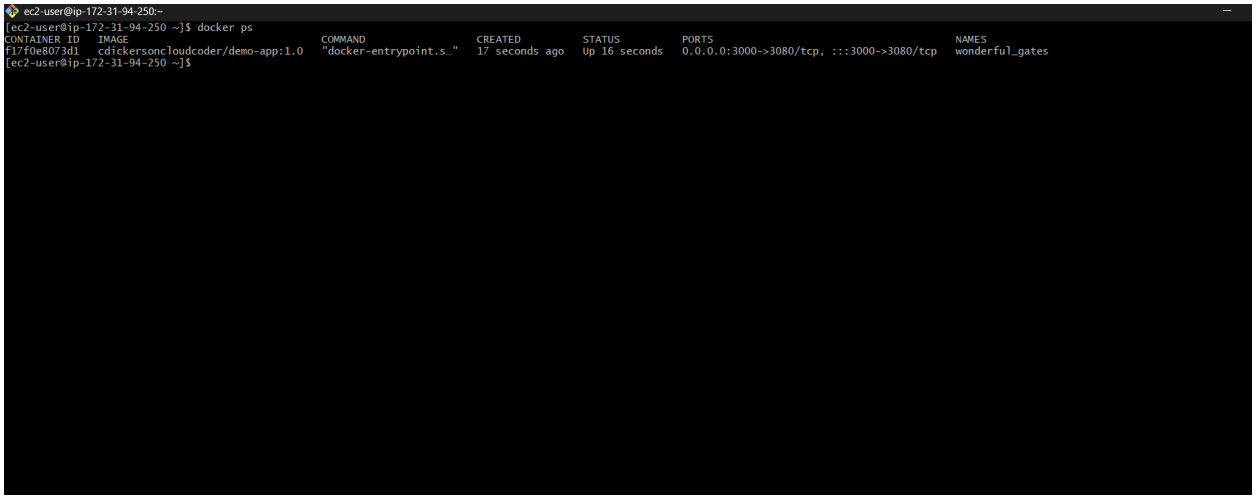
```
[ec2-user@ip-172-31-94-250 ~]$ docker run -p 300:3080 -d cdickersoncloudcoder/demo-app:1.0
a0072b5f656c5823c03da2966d3bc7ec040df44e5bec97507602e52a3a4c48c8
[ec2-user@ip-172-31-94-250 ~]$
```

💡 Launches the container in detached mode (`-d`), maps EC2 port `300` to the app's internal port `3080`, making it accessible from a browser or external request.

📂 **Verify Running Containers:**

bash
CopyEdit

```
docker ps
```



💡 Confirms the container is up and running, showing container ID, image name, status, and port mappings.

🔍 **What We Learned:**

- Successfully pulled and launched a Docker image hosted on Docker Hub.
- Gained experience with port mapping to expose containerized apps on EC2.
- Verified container status using Docker CLI.

✅ **Summary:**
You've just deployed a custom Docker application from Docker Hub to your EC2 instance. This highlights how easy it is to distribute and launch containerized apps in the cloud, enabling rapid and consistent deployments.

🔐 **Step 5: Modifying EC2 Security Group to Enable Public Access on Port 3000**

🔹 **Step Overview:**
In this step, we updated the EC2 instance's security group to allow inbound traffic on port 3000. This makes the Dockerized web app publicly accessible through a web browser.

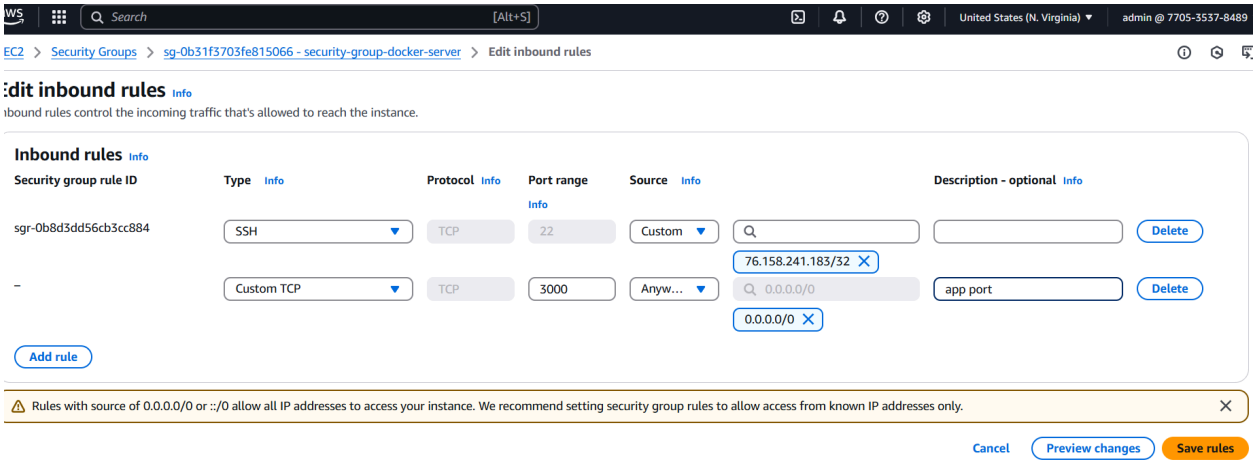
⚙️ **Actions Taken:**

📁 **Access Security Group Settings:**

🔗 [Modify Inbound Security Group Rules]
💡 Navigated to the AWS Console to configure the security group attached to the EC2 instance.

📂 **Add Inbound Rule for Port 3000:**

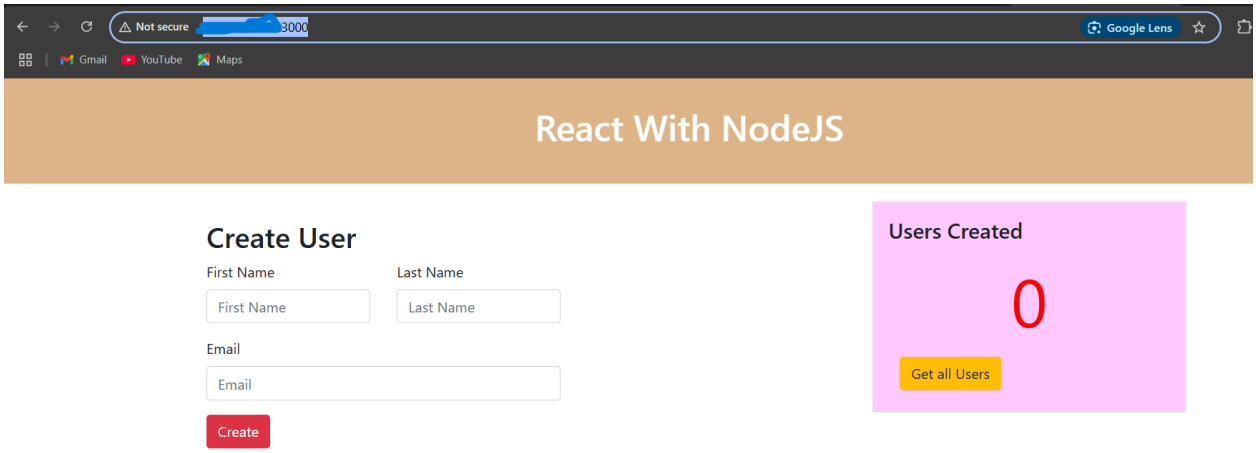
- **Type:** Custom TCP
- **Port Range:** 3000
- **Source:** 0.0.0.0/0 (Anywhere)
  💡 This rule allows traffic from any external IP to reach the application running on the EC2 instance via port 3000.



📂 **Access the Application via Public URL:**

🔗 *http://[Your-EC2-Public-IP]:3000/*
💡 Used the EC2 instance's public IP address and port 3000 to verify that the application is

reachable from the web.



🔍 **What We Learned:**

- How to securely modify EC2 security group rules to expose application ports.
- Confirmed the link between Docker port mapping and AWS firewall settings.
- Successfully accessed the app from a browser, verifying deployment.

✅ **Summary:**
You've made your Docker-hosted app accessible to the public by opening port 3000 on the EC2 security group. This is essential for hosting web services and learning cloud networking best practices.